

A Reproducible Arm-Based OpenAirInterface DU Testbed over Heterogeneous F1 Backhaul

Marc DUBOC and Ammar El Falou
King Abdullah University of Science and Technology (KAUST)
Thuwal, Saudi Arabia

Abstract—Fleet-deployable fifth-generation (5G) coverage becomes costly when every unmanned aerial vehicle (UAV) carries a complete x86 base station. We instead keep the 5G Core and central unit (CU) on the ground and define the repeated cell building block as an Arm-based OpenAirInterface (OAI) distributed unit (DU), radio, and F1 backhaul. Raspberry Pi 5 and Jetson Orin Nano DUs drive a Universal Software Radio Peripheral (USRP) B210 while F1 traverses Ethernet, Wi-Fi/Generic Routing Encapsulation (GRE), or 5G/WireGuard. Repeated end-to-end trials gate F1, registration, data, rollback, and a single-segment Public Warning System (PWS) alert displayed by a commercial handset inside a Faraday cage. Twenty trials per fixed final configuration in the same enclosed lab retain per-run observations; mean downlink is 99.8 Mbit s^{-1} on tuned Ethernet, 76.7 Mbit s^{-1} over x86 cellular backhaul, and 68.4 Mbit s^{-1} on Jetson cellular backhaul. Scheduler traces identify a Block Error Rate (BLER) adaptation mismatch as the dominant observed limiter; changing only the controller thresholds moves the dominant Modulation and Coding Scheme (MCS) from 5 to 24–27. A 106-PRB Raspberry Pi resource sample uses about 1.8 CPU cores and 1.4 GB RAM at 64.8°C without late-packet or overflow markers. The measured core electronics weigh 0.657 kg . The testbed validates the repeated Arm DU/radio/backhaul unit that motivates future one-CU/many-UAV networks; flight and simultaneous multi-DU operation remain future work.

Index Terms—5G NR, OpenAirInterface, CU/DU split, F1, AArch64, Jetson, SCTP, reproducibility, wireless backhaul, multi-UAV networks, aerial swarms

I. INTRODUCTION

Aerial and rapidly deployable base stations can restore local service after infrastructure failures and at temporary events. The research problem is not only coverage: every airborne cell consumes payload mass, electrical power, and acquisition budget. A monolithic next-generation Node B (gNB) repeats the complete base-station compute stack on every aircraft, so fleet cost grows with the number of cells. Our objective is therefore to minimize the *incremental* equipment replicated per cell while retaining a modifiable, standards-based 5G New Radio (5G NR) path to a commercial terminal.

At fleet scale, the target is a repeatable per-cell unit, not merely a smaller one-off base station. The split removes CU-side RRC/PDCP/SDAP and x86 compute from every additional aircraft, leaving the Arm DU, radio, and backhaul as the replicated cell building block. One-DU validation cannot establish multi-cell capacity, but it can establish the exact end-to-end deployment contract that a fleet would repeat; that contract is the object evaluated here.

F1 is the appropriate functional boundary for that objective. Third Generation Partnership Project (3GPP) specifications allow one gNB central unit (gNB-CU) to serve multiple gNB distributed units (gNB-DUs). The F1 control plane (F1-C) carries the F1 Application Protocol (F1AP) over the Stream Control Transmission Protocol (SCTP); the F1 user plane (F1-U) carries the user-plane General Packet Radio Service Tunnelling Protocol (GTP-U) over the User Datagram Protocol (UDP) [1], [2]. This split keeps the latency-sensitive Medium Access Control (MAC), Radio Link Control (RLC), and physical layer (PHY) beside the radio while sharing higher layers and the 5G Core (5GC). Unlike a lower-layer fronthaul, F1 does not require a purpose-built radio unit or tight transport timing. It therefore combines per-cell airborne processing with an Internet Protocol (IP) backhaul that can traverse heterogeneous links.

We use OAI rather than a proprietary stack because its publicly inspectable 5G implementation already supports the B210 real-radio path and the 3GPP CU/DU split, and because source access is essential to complete the warning path across F1 and instrument scheduler decisions. OAI also preserves an existing x86 rollback baseline, so the Arm and transport experiments change one dimension at a time. Replacing the stack would require rebuilding those baselines and the warning path before addressing the airborne question. OAI deployments have conventionally used general-purpose x86 systems [3]; moving only the DU to lower-mass 64-bit Arm compute tests whether the F1 boundary produces a practical payload benefit.

Three implementation problems still dominate: an embedded host must sustain real-time radio processing rather than merely compile OAI; embedded Linux distributions can omit SCTP, which is mandatory for F1-C; and a working F1 association can coexist with poor radio scheduling, wrong-interface traffic, or no usable handset service.

We address these problems with an evidence-driven OAI testbed. A ground 5GC/CU is paired with either a Raspberry Pi 5 or Jetson Orin Nano DU and a B210. The Pi provides the lowest-cost, lowest-mass entry point; the Jetson provides more compute headroom and matches the target embedded platform. Ethernet is kept as an intermediate deterministic wired bearer and rollback baseline. The laboratory link is copper, but the same packetized F1 case can be implemented over optical Ethernet and fibre for longer fixed segments; it is not proposed as an airborne tether. Wi-Fi/GRE creates an observable wireless route, while a Quectel 5G modem

with WireGuard provides stable, protected F1 endpoints over a mobile packet session. The workflow separately gates F1 packet placement, radio-frequency (RF) readiness, PWS reception, registration, Protocol Data Unit (PDU) session, Internet reachability, throughput, and rollback.

Previous work implemented SIB8 warning transmission in a modified monolithic OAI gNB and identified CU/DU support as future work [4]. This paper makes the tested request path functional across F1. Its contributions are:

- 1) a tested single-segment OAI PWS request path across F1 in which the CU issues F1AP warning control, the DU schedules SIB8, and a commercial handset displays the alert, tied to safe input values, the pinned OAI revision, the released patch, and a sanitized CU–F1–DU–UE execution record;
- 2) real-radio OAI 5G split-DU operation on both Raspberry Pi 5 and Jetson Orin Nano, including native AArch64 compilation and a tested, rollback-safe SCTP-enabled Jetson kernel;
- 3) an instrumented diagnosis of an OAI Modulation and Coding Scheme (MCS) collapse: scheduler traces identify a Block Error Rate (BLER) controller mismatch as the dominant observed limiter and show recovery after BLER retuning; and
- 4) a fleet-oriented testbed contract evaluated across x86, two Arm hosts, and three F1 bearers: pinned software, path and handset gates, rollback, resource observations, and payload sizing define the COTS unit replicated behind a shared CU.

All 20 repetitions per setup used the same enclosed RF environment, access profile, handset, OAI pin, and workflow. Host, bearer, and named runtime or BLER changes distinguish setups. All 180 DL observations are retained; Section V reports means and dispersion as configuration feasibility, not intrinsic bearer speed. We validate one DU at a time: simultaneous multi-DU CU capacity, inter-cell coordination, and swarm flight are enabled research directions, not results claimed by this paper. The multi-UAV use case is therefore a deployment requirement on the repeated unit, not a flight result.

II. RELATED WORK AND POSITIONING

A. From complete aerial cells to an incremental DU

Early open aerial radio access network (RAN) testbeds established feasibility by lifting most or all base-station computation. Flying Rebots placed an OAI Long-Term Evolution (LTE) evolved Node B (eNB) on commodity x86 compute aboard a custom airframe weighing roughly 2 kg before the communications payload [5]. SkyRAN carried an OAI LTE eNB and Evolved Packet Core (EPC) on separate Intel i7 and J1900 single-board computers, together with a B210 radio chain, on a heavy-lift industrial hexacopter [6]. SkyCell later paired an Intel NUC and B210 with an srsLTE eNB on the same aircraft class [7]. These designs tied the airframe to the weight, power, and cost of an x86 cellular site carried aloft.

The closest predecessor is Mundlamuri *et al.*, who already demonstrated an OAI 5G aerial DU, a B200mini, a Quectel 5G

modem, wireless F1 backhaul, and approximately 30 Mbit s⁻¹ to commercial terminals [8]. Accordingly, the novelty claimed here is neither the aerial DU concept nor wireless F1. That publication does not report the DU compute host, aircraft model, payload power or mass, Arm enablement, PWS delivery, or a cross-bearer benchmark. Our differentiation is a replication contract: the incremental DU is named, pinned, measured, exercised across heterogeneous paths, and paired with explicit service gates and rollback.

B. Arm execution and public-warning delivery

OAI is an open fourth-generation (4G)/5G implementation commonly evaluated on x86 commodity compute [3]. OAI subsequently added AArch64 compilation through the single instruction, multiple data (SIMD) portability layer SIMD Everywhere (SIMDe) [9], but source portability alone does not prove that an embedded board can sustain a real-time PHY and Universal Serial Bus (USB) radio. A 2024 6GSPACE report compiled OAI on a Raspberry Pi 4 and exercised the RF simulator, while recording unresolved runtime and architecture limitations; it did not demonstrate a real-radio 5G split DU [10]. The literature reviewed does not document the same real-radio split-DU path on both Raspberry Pi 5 and Jetson Orin Nano. Our contribution includes the system work hidden by a nominally successful compile: bounded build memory, Arm Neon/SIMDe execution, USB and central processing unit (CPU) tuning, and, on Jetson, an SCTP-capable kernel with a rollback path.

PWS literature has exposed security weaknesses using a commercial-stack testbed [11]. More recent work added System Information Block 8 (SIB8) warning transmission to a modified monolithic OAI gNB, explicitly leaving disaggregated CU/DU support for future work [4]. The present work extends that monolithic implementation: F1AP standardizes the Write-Replace Warning procedure between CU and DU [2], but the OAI request path still had to be completed and validated. In our testbed the CU sends a single-segment warning over F1-C, the remote DU schedules SIB8, and a commercial phone displays the alert. The tested profile uses identifier 1112, serial FF00, data-coding value 48, and mode 0. The artifact maps the released patch and its digest to a sanitized CU–F1–DU–UE record on the pinned OAI revision. We do not claim the response, deduplication, cancel, or multi-segment portions of the complete procedure. This matters operationally because a PWS observation simultaneously exercises CU logic, F1 signalling, DU scheduling, RF transmission, and user equipment (UE) behavior.

C. Failure diagnosis and benchmark gap

OAI documents a BLER-driven controller that raises or lowers MCS around configured thresholds [12]. Prior aerial demonstrations report a working configuration or a throughput point, but not the case encountered here: healthy F1/SCTP and UE attachment with downlink trapped at 12–22 Mbit s⁻¹ because the filtered BLER remained outside the default adaptation window. The symptom appeared during the Ethernet ex-

TABLE I
MASS AND DOWNLINK POSITIONING AGAINST REPRESENTATIVE OPEN AERIAL CELLULAR TESTBEDS. HETEROGENEOUS BANDWIDTHS, RADIOS, TERMINALS, AND TRAFFIC PREVENT A CONTROLLED RANKING.

System	Airborne RAN and compute	Reported airborne mass	Reported downlink outcome	Position relative to this work
Flying Rebots [5]	OAI LTE eNB; commodity x86 compute and USRP	2 kg airframe before communications; payload not reported	About 6 Mbit s^{-1} through relay versus 4 Mbit s^{-1} direct	Complete LTE eNB aloft; no 5G F1 split, Arm DU, or PWS path.
SkyRAN [6]	OAI LTE eNB and EPC; Intel i7 and J1900 boards; B210 chain	Communications payload not reported	$0.9\text{--}0.95\times$ the measured optimum	Complete access and core stack aloft; mass not itemized.
SkyCell [7]	srsLTE LTE eNB; Intel NUC; B210	Communications payload not reported	Up to 35% throughput improvement	Mobility-focused LTE platform; no OAI 5G, F1, or PWS.
Mundlamuri <i>et al.</i> [8]	OAI 5G DU; host not reported; B200mini and 5G modem	Not reported	30 Mbit s^{-1}	Closest aerial DU and wireless F1 result; no reported Arm build, mass, or PWS.
This work	OAI 5G DU; Raspberry Pi 5 or Jetson Orin Nano; B210 and 5G modem	0.657 kg measured core	99.8 mean Ethernet; 76.7 x86 and 68.4 Jetson mean cellular, each $n = 20$	Itemized incremental-DU mass; flight pending.

periment, but the adaptation mechanism is bearer-independent: it can occur whenever the radio-link BLER remains outside the configured window. Section V provides the observed BLER mechanism and the paired before/after intervention. The reported throughput remains an end-to-end result of the complete validated configuration rather than a universal scheduler optimum.

Unlike prior single-platform demonstrations, our public COTS matrix crosses x86, Raspberry Pi 5, and Jetson; monolithic and split execution; and three F1 bearers. Exact software, packet-path evidence, handset gates, rollback, and an itemized mass envelope make the prototype reproducible without claiming unmeasured flight endurance.

III. TESTBED AND REPRODUCIBILITY METHOD

A. Architecture

Fig. 1 shows the system. The ground host runs the OAI 5GC and CU. The CU terminates Radio Resource Control (RRC), Packet Data Convergence Protocol (PDCP), and Service Data Adaptation Protocol (SDAP); the selected Arm DU runs the MAC/RLC/PHY layers defined above and drives a B210 over USB 3. The access cell uses band n78, 30-kHz subcarrier spacing, and 106 Physical Resource Blocks (PRBs). A commercial handset is the end-to-end UE. PWS adds an observable application path: the CU issues an F1AP Write-Replace Warning request, the DU schedules SIB8, and the handset displays the warning [2].

The figure shows the DU validated in this work as DU_1 . From the CU's point of view, the same F1 architecture can fan out to $\text{DU}_2 \dots \text{DU}_N$, each with its own F1 endpoints, access cell, and airborne payload. Replicating the green payload while retaining the ground 5GC/CU is the intended multi-UAV extension; the dashed branch is architectural scale-out, not a simultaneous experiment.

The B210 provides up to 56 MHz real-time bandwidth and a 61.44-mega-sample-per-second quadrature path over Super-Speed USB 3 [13]. The Quectel case uses two independent cells: a donor cell serves the modem, while the B210 access cell serves the handset. WireGuard gives stable F1 endpoint addresses over the modem's dynamic packet session. This separation prevents the common false positive in which the

handset joins the donor cell while the experiment is attributed to the access DU.

B. Artifact contract and security boundary

The public artifact [14] tracks the required docs/, safe PWS example, and patches/. Its manifest gives the full OAI revision and SHA-256 patch digests; the PWS patch is patches/sib8/oai-pws-sib8-cu-du.patch. A sanitized record maps it to F1AP handling, DU scheduling, handset observation, rollback, the tested warning values, and safe log fingerprints. Pinned historical records corroborate the configuration lineage and early GRE and 5G/WireGuard gates but are not artifact inputs. Runtime inputs, raw logs/captures, credentials, database identifiers, and keys are excluded. Prior exposed test values were revoked and replaced, and publication checks cover Git history and the current tree.

The access B210 and intended handset operate inside the lab Faraday cage on an isolated test public land mobile network (PLMN), using explicitly test-only warning text. A run is invalid if radio containment, the intended UE, the selected access cell, or exclusive lab ownership cannot be proven.

The operator workflow enforces the sequence below:

- 1) discover the selected host, B210, modem, addresses, routes, USB speed, OAI commit, and patch state;
- 2) acquire an exclusive operator lock, generate the scenario config, and launch the 5GC, CU, transport, and DU in dependency order;
- 3) prove core health, F1-C SCTP, F1-U GTP-U, RF synchronization, and the absence of F1 leakage onto management interfaces;
- 4) separately record handset PWS, registration, PDU session, external data network (DN) reachability, Internet access, downlink/uplink throughput, and timestamps;
- 5) stop all processes, check residue, and reproduce the Ethernet rollback.

This distinction matters: F1 setup and an RF-ready message are machine-side milestones, not evidence of handset service. A scenario is accepted only when the relevant phone-visible gates also pass.

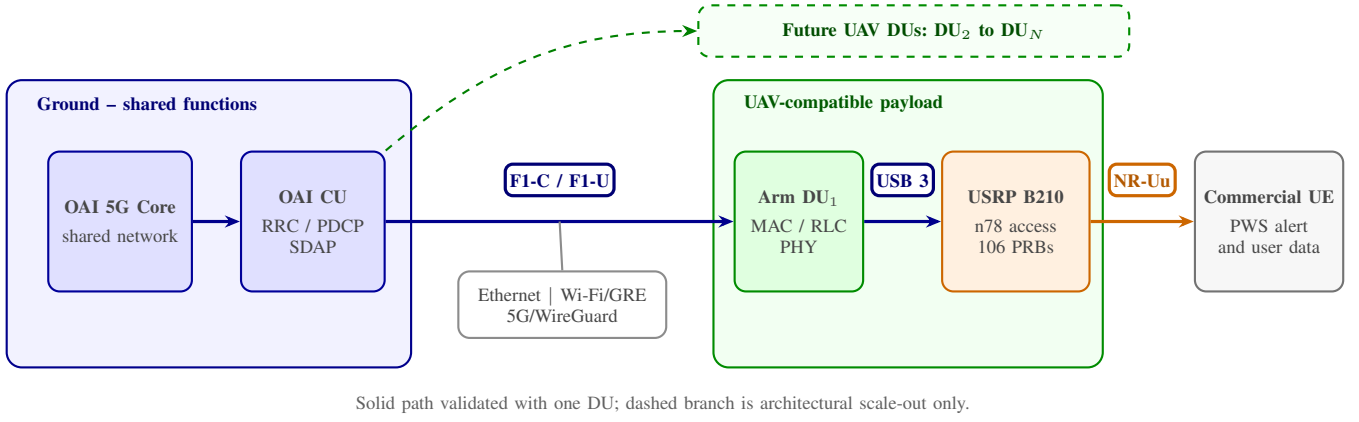


Fig. 1. Shared-CU testbed architecture. The solid DU_1 path is validated; the dashed $DU_2 \dots DU_N$ branch shows the multi-UAV scale-out direction. Management traffic stays outside the selected F1 bearer, and packet captures verify the actual path.

IV. NATIVE AARCH64 AND JETSON ENABLEMENT

A. Building OAI on Arm

The working method is a native build on the target, not an x86 binary copied to the board. OAI’s SIMD-backed code maps the x86-style vector application programming interface (API) used throughout the PHY onto AArch64/Neon-compatible implementations [9]. We freeze the same OAI commit on CU and DU, install dependencies on the Arm host, and build only the required gNB/USRP target:

```
git checkout \
  102965a669b9444857c27843ec8ce62780bf9d37
cd cmake_targets
sudo ./build_oai -I
sudo ./build_oai -w USRP --ninja --gNB -C
```

Four details made this repeatable. The `libsctp-dev` and `lksctp-tools` packages are installed before the build so compile-time headers and runtime diagnostics agree. The USRP Hardware Driver (UHD) and its B210 images are installed locally; the USB radio cannot be treated as a remote x86 peripheral. Build parallelism is bounded and swap is available because an 8-GB board can otherwise fail during compilation or linking. Finally, the resulting `nr-softmodem`, shared parameter library, UHD backend, and exact source identity are checked on the target. No Jetson-specific OAI source fork is required for AArch64 itself; the lab’s PWS and instrumentation changes remain explicit patches on the pinned tree.

B. Why the stock Jetson kernel failed

The Jetson Orin Nano ran NVIDIA Linux for Tegra (L4T) R36.4.4 / JetPack 6.2 with kernel 5.15.148-tegra. The stock configuration contained `# CONFIG_IP_SCTP is not set`; `modprobe sctp` therefore failed, and creating an SCTP socket returned “protocol not supported.” This blocks both F1-C and any local F1 validation before radio processing starts.

A stand-alone `sctp.ko` built from generic or mismatched headers was not a safe fix. NVIDIA’s Board Support Package (BSP) uses out-of-tree Graphics Processing Unit (GPU), display, audio, and Tegra drivers. If the main image, module tree, compiler/configuration, and `CONFIG_LOCALVERSION` disagree, modules fail symbol/version checks; if the out-of-tree set is omitted, the kernel may boot while graphics and other board functions fail.

The reproducible fix is published as a separate build artifact [15]:

- 1) inventory the exact L4T package and stock boot entry;
- 2) download the matching R36.4.4 public sources, including in-tree, NVIDIA out-of-tree, and display-driver trees;
- 3) enable `CONFIG_IP_SCTP=m` and `CONFIG_INET_SCTP_DIAG=m`, and assign an isolated local version;
- 4) compile natively with a temporary 4GB swap file and parallelism limited to four jobs, then stage the image and coherent modules;
- 5) generate a matching `initrd` and add, rather than overwrite, an `extlinux.conf` entry so the stock kernel remains selectable.

After reboot, `checksctp`, a Python SCTP socket, and `loopback sctp_darn` all passed; GPU/display and Ethernet drivers remained loaded. This validation is as important as the successful kernel build.

C. Real-time runtime profile

Kernel support alone did not make the Jetson a stable DU. At USB 2 speed (480 Mbit/s), the B210 produced continuous UHD receive overflows. A real SuperSpeed path at 5000M was necessary but still insufficient. The validated runtime sets maximum-performance mode, locks clocks, disables USB autosuspend, sets `usbfs_memory_mb=1000`, runs the DU on CPUs 1–5, and pins the *active* extensible Host Controller Interface (xHCI) interrupt to CPU 0. Selecting the active

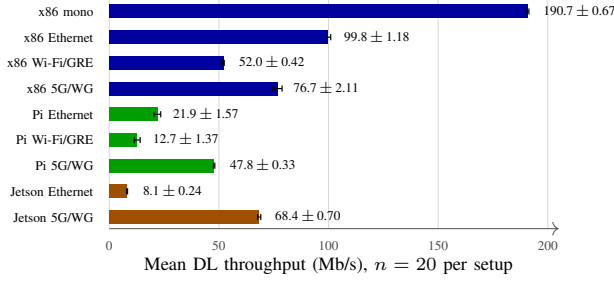


Fig. 2. Mean handset DL ($n = 20$); labels and whiskers show 95% CI half-widths. The artifact retains all 180 observations.

interrupt by increasing counters, rather than an arbitrary USB-looking line in `/proc/interrupts`, removed a repeatability trap. Warning-level OAI logging reduces avoidable real-time pressure.

V. BENCHMARKS AND MCS-COLLAPSE DIAGNOSIS

A. Measurement protocol

Every supported scenario was exercised in repeated end-to-end trials. The Faraday cage, intended handset, n78 profile (30-kHz spacing, 106 PRBs), OAI pin, and gate sequence were fixed; host and F1 bearer were the named variables. A trial required the intended access cell, PWS, registration, PDU session, Internet, clean stop, and rollback. Captures established whether F1-C/F1-U used Ethernet, GRE, or WireGuard; the cellular case also required encrypted modem-interface traffic. DU logs supplied MCS, exponentially filtered BLER, Hybrid Automatic Repeat Request (HARQ) rounds, signal-to-noise ratio (SNR), Channel Quality Indicator (CQI) caps, and overflow counts. The handset browser on <https://fast.com> measured downlink (DL) and uplink (UL) rates. The artifact retains all 180 DL observations (20 per final configuration). Two-sided 95% confidence intervals (CIs) use the sample standard deviation and Student's $t_{0.975,19} = 2.093$. The fixed lab bounds environmental variation, but host, bearer, runtime, and BLER differences preclude a one-factor ranking.

The means establish feasibility across Arm platforms and bearers without showing that a tunneled cellular path is intrinsically faster than Ethernet. The 68.4 Mbit s^{-1} Jetson 5G/WireGuard mean used the full MCS range and validated runtime state.

B. Observed mechanism

The initial Ethernet split reached at most about 23 Mb/s with MCS at its configured floor of 5, although the same access radio could reach high MCS in other scenarios. Ethernet identifies the experiment in which the symptom was isolated; it is not the cause. Instrumentation of `get_mcs_from_bler()` showed OAI applying the filtered estimate

$$\hat{b}_k = 0.9\hat{b}_{k-1} + 0.1b_k, \quad (1)$$

then incrementing MCS only when $\hat{b}_k < b_{\text{lower}}$ and decrementing it when $\hat{b}_k > b_{\text{upper}}$. The source defaults

TABLE II
BLER-THRESHOLD INTERVENTION AND FINAL MEAN THROUGHPUT

Metric	Before	After
DL target window	0.05–0.15	0.25–0.35
Dominant MCS	5	24 and 27
Observed DL BLER	22–35%	about 22%
Handset DL	up to 23 Mb/s	99.8 Mb/s mean ($n = 20$)
External-DN round-trip time	up to seconds	11–30 ms

were $(b_{\text{lower}}, b_{\text{upper}}) = (0.05, 0.15)$. Under sustained Ethernet traffic, the measured link showed 22–35% initial-round HARQ retransmissions and windowed BLER commonly between 0.18 and 0.40. The 5% increment condition was therefore effectively unreachable, so the controller repeatedly reduced MCS to the floor.

The direct evidence therefore identifies the BLER-controller mismatch, not Ethernet capacity, as the dominant observed limiter. The same controller behavior can arise on any bearer when its access radio shows a comparable BLER distribution.

C. BLER retuning and bounded interpretation

In the after condition, we configured the DU with $b_{\text{DL,lower}} = 0.25$, $b_{\text{DL,upper}} = 0.35$, $b_{\text{UL,lower}} = 0.15$, and $b_{\text{UL,upper}} = 0.35$. The radio profile, attenuation, numerology, Bandwidth Part (BWP), F1 path, UPF, and transport settings were unchanged. Table II records the single changed factor and the observed response.

In a five-minute window after restart, MCS 24 and 27 were most frequent; the scheduler was BLER-limited, not MCS-table-capped. The retuned setup averaged 99.8 Mbit s^{-1} ($n = 20$). The wider target trades HARQ retransmissions for spectral efficiency in this RF condition; the thresholds are exposed as scenario parameters.

Filtered BLER above target followed by MCS reduction identifies the dominant radio-layer mechanism, and the threshold-only intervention links controller direction to MCS recovery under the recorded RF condition. This is a configuration-specific result, not a universal BLER target recommendation.

VI. EMBEDDED RESOURCES AND PAYLOAD MASS

Sanitized resource records in the artifact [14] add host-level evidence. A 51-PRB Raspberry Pi 5 sample recorded load 2.04, 1.5 GiB used, 67.0°C , no throttling, and no UHD underruns. At 106 PRBs, another sample used about 1.8 cores and 1.4 GB RAM at 64.8°C , with no late-packet or overflow markers after thread pinning and with PWS passing. These are synchronized samples, not cross-platform averages.

No aircraft has yet carried the testbed, so the mass model is anchored to the Jetson, B210, and Quectel configuration that passed attach, PWS, user-plane, and rollback gates. Table III separates measured electronics from minimum integration allowances; the allowance yields a 0.757 kg integration estimate.

TABLE III
MEASURED CORE MASS AND B200MINI BOARD-ONLY PROJECTION

Item	Mass (g)
Jetson Orin Nano board	151.4
USRP B210 with access antennas	217.3
Quectel modem kit with antennas	288.7
Measured core electronics	657.4
Regulator, harness, mounting allowance	100.0
Estimated integration with allowance	757.4
B200mini board in place of B210 assembly	24.0
Projected mini-radio integration*	564.1

*Upper-bound mass reduction before adding mini-radio antennas and enclosure; the B200mini path has not undergone end-to-end validation.

Ettus specifies a 24 g board-only mass for the B200mini, compared with the much larger B200/B210 form factor [13]. Substituting that board for the measured 217.3 g B210-and-antenna assembly gives $757.4 - 217.3 + 24.0 = 564.1$ g. This is deliberately reported as a projection: antennas, protection, mounting, and validation can reduce the 193.3 g upper-bound saving. It also explains why the validated B210 value, rather than the lighter untested radio, remains the headline.

With the engineering rule that payload mass occupy at most 70% of advertised capacity, the estimated integration requires at least $0.757/0.70 = 1.08$ kg of payload rating. The mini-radio projection would start at 0.81 kg before its missing allowances. Vehicle selection still requires battery-input, thermal, vibration, center-of-gravity, electromagnetic-compatibility, and failsafe tests.

VII. CONCLUSION

Repeated end-to-end trials in one fixed enclosed lab environment validated handset-visible single-segment PWS and data across Ethernet, Wi-Fi/GRE, and 5G/WireGuard F1 bearers. Jetson delivered mean 68.4 Mbit s^{-1} over 5G/WireGuard ($n = 20$). Changing only the BLER thresholds moved the dominant MCS from 5 to 24–27; the retuned Ethernet configuration averaged 99.8 Mbit s^{-1} ($n = 20$). Core electronics weigh 0.657 kg; the 0.757 kg integration estimate requires a 1.08 kg payload rating, while an unvalidated B200mini substitution projects 0.564 kg. Raspberry Pi resource observations provide compute and thermal headroom evidence. The Arm DU/radio/backhaul payload, pinned software, path and handset gates, and rollback form the replicated cell building block. Keeping CU-side functions on the ground makes this bounded unit—not a complete x86 base station—what fleet size multiplies. A uniform cross-setup matched-condition factorial comparison, measured payload power, concurrent DUs, and controlled flight remain future work.

REFERENCES

- [1] 3GPP, “TS 38.401: NG-RAN; architecture description,” 3rd Generation Partnership Project, Technical Specification, 2025, release 18, version 18.6.0. [Online]. Available: <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3219>
- [2] —, “TS 38.473: NG-RAN; F1 application protocol (F1AP),” 3rd Generation Partnership Project, Technical Specification, 2026, release 18, version 18.10.0. [Online]. Available: <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3260>
- [3] F. Kaltenberger, A. P. Silva, A. Gosain, L. Wang, and T.-T. Nguyen, “OpenAirInterface: Democratizing innovation in the 5G era,” *Computer Networks*, vol. 176, p. 107284, 2020.
- [4] A. Abou Hasna, N. Chendeb, and A. El Falou, “From spoofing to trust: Emergency alerts spoofing testbed and cross-cell verification,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.24404>
- [5] R. Gangula, O. Esrafilian, D. Gesbert, C. Roux, F. Kaltenberger, and R. Knopp, “Flying rebots: First results on an autonomous UAV-based LTE relay using OpenAirInterface,” in *Proc. IEEE International Workshop on Signal Processing Advances in Wireless Communications (SPAWC)*, 2018, pp. 1–5.
- [6] A. Chakraborty, E. Chai, K. Sundaresan, A. Khojastepour, and S. Rangarajan, “SkyRAN: A self-organizing LTE RAN in the sky,” in *Proc. ACM Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2018, pp. 280–292.
- [7] L. Ferranti, L. Bonati, S. D’Oro, and T. Melodia, “SkyCell: A prototyping platform for 5G aerial base stations,” in *Proc. IEEE WoWMoM*, 2020, pp. 329–334.
- [8] R. Mundlamuri, O. Esrafilian, R. Gangula, R. Kharade, C. Roux, F. Kaltenberger, R. Knopp, and D. Gesbert, “Integrated access and backhaul in 5G with aerial distributed unit using OpenAirInterface,” in *Proc. 17th ACM Workshop on Wireless Network Testbeds, Experimental Evaluation and Characterization (WiNTECH)*, 2023, demo. [Online]. Available: <https://www.eurecom.fr/publication/7304>
- [9] OpenAirInterface Software Alliance, “OpenAirInterface5G v2.1.0 release notes: ARM compilation through SIMDe,” <https://gitlab.eurecom.fr/oai/openairinterface5g/-/tags/v2.1.0>, 2024, accessed: 2026-07-20.
- [10] SnT/SIGCOM 6GSPACE Lab, “Past and current activities on 5G and 6G NTN using OAI at the 6GSPACE lab of SnT/SIGCOM,” EURECOM/OAI NTN Meeting presentation, 2024, 5 April 2024; accessed: 2026-07-20. [Online]. Available: https://openairinterface.org/wp-content/uploads/2024/03/2024-04-05_SnT_EURECOM-OAI-NTN-Meeting.pdf
- [11] E. Bitsikas and C. Pöpper, “You have been warned: Abusing 5G’s warning and emergency systems,” in *Proc. 38th Annual Computer Security Applications Conference (ACSAC)*, 2022, pp. 561–575.
- [12] OpenAirInterface Software Alliance, “OpenAirInterface 5G MAC usage and scheduler documentation,” Project documentation, accessed: 2026-07-20. [Online]. Available: <https://github.com/OPENAIRINTERFACE/openairinterface5g/blob/develop/doc/MAC/mac-usage.md>
- [13] Ettus Research, “B200/B210/B200mini/B205mini/B206mini hardware and physical specifications,” <https://kb.ettus.com/B200/B210/B200mini/B205mini/B206mini>, accessed: 2026-07-20.
- [14] OAI CU/DU Lab Maintainers, “OAI CU/DU Lab: Reproducible 5G Split-RAN artifact,” <https://github.com/promaaa/oai-cu-du-lab>, 2026, accessed: 2026-08-13; OAI pin, feature patches, operator workflow, and sanitized evidence.
- [15] promaaa, “Jetson Orin Nano custom SCTP kernel and OOT drivers guide,” <https://github.com/promaaa/jetson-kernel-sctp>, 2026, accessed: 2026-07-20.